1 Overview (Coding Standard)

- Prefer `constexpr` for compile-time evaluation and `consteval` when a function *must* be evaluated at compile time; do not force compile-time work by macro tricks.
- Use `const` to express immutability at the API boundary; prefer `const` member functions for logical constness; avoid `mutable` except for caching or synchronization.
- Prefer `static_cast` for well-defined, explicit conversions; reserve `reinterpret_cast` for low-level, representation-oriented code behind narrow interfaces.
- Use `noexcept` on move operations and leaf functions where you can uphold it; never add `noexcept` as a "performance hint" unless it is true.
- Use `auto` when it improves correctness (e.g., avoids narrowing, keeps ref/const qualifiers via `auto&`/`auto&&`); avoid `auto` when it hides a meaningful type in interfaces.
- Prefer `enum class` over unscoped `enum` to avoid implicit conversions and name pollution.
- Use `switch` with `default` intentionally: either handle all cases explicitly (no `default`) or include `default` as a deliberate catch-all.
- In templates, constrain with `requires` / `concept` rather than SFINAE-era workarounds; keep constraints readable and close to the declaration.
- Avoid `goto`; use structured control flow (`if`, `for`, `while`) and `break`/`continue` with restraint.
- Do not use `register` (deprecated); do not rely on `volatile` for concurrency.

2 Keyword Cheat Sheet (Alphabetical)

2.1 `alignas`

- **Meaning:** Specifies a minimum alignment requirement for an object or type.
- **Where it applies:** declaration; type.
- **Use when:**
  - You need stronger-than-default alignment for performance or ABI requirements.
  - You wrap low-level storage that must satisfy a type's alignment.
- **Avoid when:**
  - You are guessing alignments instead of using `alignof`.
  - You cannot guarantee the target environment supports the requested alignment.
- **See also:** `alignof`, `decltype`.

Example:

```
struct alignas(64) CacheLine { int x; };
CacheLine cl{};
```

Gotchas:

- Over-alignment can increase size/padding and affect layout/ABI.

2.2 `alignof`

- **Meaning:** Yields the alignment requirement of a type or expression's type.
- **Where it applies:** expression.
- **Use when:**
  - You need to compute or validate storage alignment.
  - You pair with `alignas` to document requirements.

- **Avoid when:**
  - You hardcode alignment constants without justification.
  - You confuse alignment with size (`sizeof`).
- **See also:** `alignas`, `sizeof`.

Example:

```
static_assert(alignof(long double) >= alignof(double));
```

Gotchas:
- Alignment depends on the target ABI and can vary by platform.

2.3 `and`

- **Meaning:** Alternative token for `&&` (logical AND).
- **Where it applies:** expression.
- **Use when:**
  - You prefer word operators for readability in boolean expressions.
  - You work in contexts where `&&` is visually noisy.
- **Avoid when:**
  - Your codebase standardizes on symbolic operators.
  - Mixing word and symbolic operators reduces consistency.
- **See also:** `or`, `not`.

Example:

```
bool ok = ready and valid;
```

Gotchas:
- Operator precedence is the same as `&&`.

2.4 `and_eq`

- **Meaning:** Alternative token for `&=` (bitwise AND assignment).
- **Where it applies:** expression.
- **Use when:**
  - You prefer word operators for bitmask manipulations.
  - You want to avoid punctuation-heavy code.
- **Avoid when:**
  - Your team expects `&=` for bit operations.
  - It harms grep/search familiarity with common idioms.
- **See also:** `bitand`, `or_eq`, `xor_eq`.

Example:

```
unsigned flags = 0b1111u;
flags and_eq 0b0110u; // keep middle bits
```

Gotchas:
- This is bitwise, not logical; do not confuse with `and`.

2.5 `asm`

- **Meaning:** Introduces an implementation-defined assembler statement.
- **Where it applies:** statement; declaration (compiler extension behavior varies).
- **Use when:**
  - You need a tiny, well-audited platform-specific instruction sequence.
  - You are writing a low-level boundary (e.g., context switch) with no alternative.
- **Avoid when:**
  - A standard C++ solution exists (portability and tooling matter).
  - You cannot prove correctness across optimization levels.

- **Notes (C++20):** Semantics are implementation-defined; inline asm is not standardized.

Example:

```
void fence() { asm volatile("" ::: "memory"); } // compiler barrier (non-portable)
```

Gotchas:
- Inline asm constraints/clobbers are compiler-specific and easy to get wrong.

2.6 `atomic_cancel`

- **Meaning:** Reserved keyword related to transactional memory (TM) control.
- **Where it applies:** (reserved; not used in standard C++20 programs).
- **Use when:**
  - Only in implementations that support TM extensions using these reserved words.
- **Avoid when:**
  - Writing portable C++20 (treat as unavailable).
  - Naming identifiers (cannot be used as an identifier).
- **Notes (C++20):** Present as a reserved keyword for possible TM support; not part of core portable idioms.

Example:

```
// Not portable C++20: reserved for TM extensions.
```

Gotchas:
- May break code that tried to use it as an identifier.

2.7 `atomic_commit`

- **Meaning:** Reserved keyword related to transactional memory (TM) commit.
- **Where it applies:** (reserved; not used in standard C++20 programs).
- **Use when:**
  - Only with specific TM-enabled compilers/extensions.
- **Avoid when:**
  - Portability is required.
  - You want an identifier with this spelling.
- **Notes (C++20):** Reserved; do not rely on it in portable code.

Example:

```
// Reserved keyword; no standard portable usage.
```

Gotchas:
- Treat as a keyword in tokenization even if unused.

2.8 `atomic_noexcept`

- **Meaning:** Reserved keyword related to transactional memory (TM) "noexcept"-like control.
- **Where it applies:** (reserved; not used in standard C++20 programs).
- **Use when:**
  - Only for TM extensions that define meaning for it.
- **Avoid when:**
  - Writing portable C++20; use `noexcept` for exceptions instead.
  - Naming variables or functions with this token.
- **See also:** `noexcept`.
- **Notes (C++20):** Reserved; unrelated to standard exception specifications.

Example:

```
// Reserved keyword; no standard portable usage.
```

Gotchas:
- Do not confuse with `noexcept`.

2.9 `auto`

- **Meaning:** Deduces a type from an initializer (or serves as a placeholder in some contexts).
- **Where it applies:** declaration; function return type; template parameter (abbreviated forms); lambda.
- **Use when:**
  - The initializer makes the type obvious and reduces repetition.
  - You want to preserve qualifiers intentionally via `auto&` / `const auto&`.
  - You want to avoid accidental narrowing by delegating to deduction rules.
- **Avoid when:**
  - It hides a semantic type important to readability (especially in APIs).
  - It deduces `std::initializer_list`-like behavior via braced init and surprises you.
- **See also:** `decltype`, `const`.

Distinct uses:

- **Variable type deduction:** Deduces from initializer; prefer `auto&` to avoid copies.
- **Function return type placeholder:** Return type deduced from `return` expressions.
- **Generic lambda parameter:** `auto` parameters create a function template.

Example:

```
auto x = 42;          // int
const auto& r = x;    // const int&
auto id = [](auto v){ return v; };
```

Gotchas:

- `auto` drops top-level `const` unless you write it explicitly.

2.10 `bitand`

- **Meaning:** Alternative token for `&` (bitwise AND).
- **Where it applies:** expression.
- **Use when:**
  - You want a word form for bitwise operations in dense expressions.
  - You are in codebases where `&` is easily confused with reference syntax.
- **Avoid when:**
  - Consistency with common C++ idioms favors `&`.
  - It reduces clarity for readers expecting symbolic operators.
- **See also:** `and_eq`, `bitor`, `xor`.

Example:

```
unsigned m = 0b1010u bitand 0b0110u; // 0b0010
```

Gotchas:

- This is bitwise AND, not address-of or reference syntax.

2.11 `bitor`

- **Meaning:** Alternative token for `|` (bitwise OR).
- **Where it applies:** expression.
- **Use when:**
  - Bitmask expressions read better as words in your codebase.
  - You want to avoid punctuation-heavy expressions.
- **Avoid when:**
  - Your team standard uses `|`.
  - Readers may confuse with logical `or`.
- **See also:** `or_eq`, `xor`.

Example:

```
unsigned m = 0b1000u bitor 0b0011u; // 0b1011
```

Gotchas:
- Do not confuse `bitor` (bitwise) with `or` (logical).

2.12 `bool`

- **Meaning:** Fundamental boolean type with values `true` and `false`.
- **Where it applies:** type.
- **Use when:**
  - You represent a true/false state.
  - You write predicate return types.
- **Avoid when:**
  - A multi-state result should be represented by `enum class`.
  - Meaning is unclear; name variables to express intent.
- **See also:** `true`, `false`.

Example:

```
bool done = false;
if (!done) { done = true; }
```

Gotchas:
- Avoid mixing booleans with integers in APIs (implicit conversions exist).

2.13 `break`

- **Meaning:** Exits the nearest enclosing loop or `switch`.
- **Where it applies:** statement.
- **Use when:**
  - You need early exit from a loop upon a condition.
  - You terminate a `case` in a `switch`.
- **Avoid when:**
  - Multiple `breaks` obscure loop logic (consider restructuring).
  - You want to exit multiple nesting levels (prefer refactoring).
- **See also:** `continue`, `return`, `switch`.

Example:

```
for (int i=0; i<10; ++i) {
  if (i == 3) break;
}
```

Gotchas:
- In a `switch`, missing `break` causes fallthrough.

2.14 `case`

- **Meaning:** Labels a branch in a `switch` statement.
- **Where it applies:** statement.
- **Use when:**
  - You dispatch on integral/enum values.
  - You want a compact multi-way branch.
- **Avoid when:**
  - Conditions are non-discrete or boolean-combinational (use `if`).
  - Fallthrough would be accidental or confusing.
- **See also:** `switch`, `default`, `break`.

Example:

```
switch (n) {
  case 0: return 1;
```

```
  case 1: return 1;
  default: return 0;
}
```

Gotchas:
- `case` labels must be constant expressions.

2.15 `catch`

- **Meaning:** Introduces an exception handler for a `try` block.
- **Where it applies:** statement.
- **Use when:**
  - You can handle a failure locally and preserve invariants.
  - You need to translate exceptions at module boundaries.
- **Avoid when:**
  - You cannot meaningfully recover (let it propagate).
  - You catch-all and silently ignore errors.
- **See also:** `try`, `throw`.

Example:

```
try { /* ... */ }
catch (const int&) { /* handle */ }
```

Gotchas:
- Prefer catching by (const) reference to avoid slicing/copies.

2.16 `char`

- **Meaning:** Fundamental character type (narrow character).
- **Where it applies:** type.
- **Use when:**
  - You manipulate raw bytes or narrow text (encoding-dependent).
  - Interfacing with C APIs expecting `char`.
- **Avoid when:**
  - You need explicit signedness (use `signed char`/`unsigned char`).
  - You assume `char` is signed or unsigned (it is implementation-defined).
- **See also:** `signed`, `unsigned`, `wchar_t`.

Example:

```
char c = 'A';
```

Gotchas:
- `char` signedness is implementation-defined.

2.17 `char8_t`

- **Meaning:** Distinct type for UTF-8 code units.
- **Where it applies:** type.
- **Use when:**
  - You need a dedicated type for UTF-8 data.
  - You want overload separation from `char`.
- **Avoid when:**
  - You rely on implicit conversion to/from `char` (not the same type).
  - You need platform-specific narrow APIs that only accept `char`.
- **Notes (C++20):** Introduced in C++20 as a distinct fundamental type.

Example:

```
char8_t u = u8'A';
```

Gotchas:
- Interop with `char` often requires explicit casts/conversions.

2.18 `char16_t`

- **Meaning:** Fundamental type for UTF-16 code units.
- **Where it applies:** type.
- **Use when:**
  - You store or process UTF-16 code units explicitly.
  - You interoperate with UTF-16-based system APIs.
- **Avoid when:**
  - You treat code units as code points (surrogates exist).
  - You need locale-dependent narrow text; prefer explicit encoding handling.
- **See also:** `char32_t`, `wchar_t`.

Example:

```
char16_t u = u'A';
```

Gotchas:
- UTF-16 may require surrogate pairs for some code points.

2.19 `char32_t`

- **Meaning:** Fundamental type for UTF-32 code units (often one code point).
- **Where it applies:** type.
- **Use when:**
  - You want fixed-width 32-bit code units for Unicode processing.
  - You interoperate with UTF-32 interfaces.
- **Avoid when:**
  - Memory footprint matters and UTF-32 is too large.
  - You assume 1:1 mapping always equals "user-perceived characters".
- **See also:** `char16_t`, `wchar_t`.

Example:

```
char32_t u = U'A';
```

Gotchas:
- Unicode grapheme clusters can span multiple code points.

2.20 `class`

- **Meaning:** Declares a class type; default member access is `private`.
- **Where it applies:** class; type; declaration.
- **Use when:**
  - You define an invariant-bearing user-defined type.
  - You want encapsulation via default `private`.
- **Avoid when:**
  - A passive aggregate-like type fits better as `struct` by convention.
  - You expose all members publicly (consider `struct` or redesign).
- **See also:** `struct`, `public`, `private`.

Example:

```
class Counter {
  int v = 0;
public:
  void inc() { ++v; }
};
```

Gotchas:
- Default inheritance access is `private` for `class`.

2.21 `compl`

- **Meaning:** Alternative token for ~ (bitwise NOT).
- **Where it applies:** expression.
- **Use when:**
  - You prefer word operators in bit manipulation.
  - ~ might be confused with destructor syntax in context.
- **Avoid when:**
  - Your codebase uses ~ consistently.
  - It surprises readers unfamiliar with alternative tokens.
- **See also:** `not`, `xor`.

Example:

```
unsigned m = compl 0u; // all bits set
```

Gotchas:
- This is bitwise complement, not logical negation.

2.22 `concept`

- **Meaning:** Declares a named compile-time predicate for template constraints.
- **Where it applies:** template; declaration.
- **Use when:**
  - You want reusable, readable constraints.
  - You want clearer diagnostics than ad-hoc SFINAE.
- **Avoid when:**
  - A one-off constraint is simpler as an inline `requires` clause.
  - You cannot define the constraint without accidental over-constraint.
- **See also:** `requires`, `template`.
- **Notes (C++20):** Concepts are a core C++20 feature.

Example:

```
template<class T>
concept Incrementable = requires(T x) { ++x; };

template<Incrementable T>
T f(T x) { return ++x; }
```

Gotchas:
- Concepts constrain templates; they do not change runtime behavior.

2.23 `const`

- **Meaning:** Qualifies an entity as non-modifiable through that name.
- **Where it applies:** type; declaration; member function.
- **Use when:**
  - You define read-only views/parameters and enforce invariants.
  - You mark member functions that do not mutate the logical state.
  - You declare pointers/references with intended immutability.
- **Avoid when:**
  - You plan to cast it away (`const_cast`) as a design pattern.
  - It falsely suggests thread-safety or deep immutability.
- **See also:** `mutable`, `const_cast`.

Distinct uses:
- **Const object:** `const T x;` makes `x` non-assignable.
- **Const pointer vs pointer-to-const:** `T* const` vs `const T*`.
- **Const member function:** `T f() const;` promises not to mutate `*this` (except `mutable`).

Example:

```
struct S {
  int x = 0;
  int get() const { return x; }
};
const int* p = nullptr;    // pointer to const int
int* const q = nullptr;    // const pointer
```

Gotchas:
- Top-level `const` is dropped by `auto` unless you write it.

2.24 `consteval`

- **Meaning:** Declares an *immediate* function that must be evaluated at compile time.
- **Where it applies:** function.
- **Use when:**
  - You require compile-time computation (e.g., validating constants).
  - You want to reject runtime use by construction.
- **Avoid when:**
  - Runtime evaluation is sometimes acceptable (use `constexpr`).
  - You need side effects or runtime-only operations.
- **See also:** `constexpr`.
- **Notes (C++20):** Introduced in C++20.

Example:

```
consteval int sq(int x) { return x * x; }
constexpr int n = sq(12);
```

Gotchas:
- Calling a `consteval` function in a non-constant context is ill-formed.

2.25 `constexpr`

- **Meaning:** Indicates that a value or function can be evaluated at compile time when used in constant contexts.
- **Where it applies:** variable; function; constructor.
- **Use when:**
  - You want compile-time constants without macros.
  - You want functions usable both at compile time and runtime.
  - You want stronger optimization opportunities with clear intent.
- **Avoid when:**
  - The function cannot reasonably meet constant-evaluation rules.
  - You need "must be compile-time" (use `consteval`).
- **See also:** `consteval`, `constinit`.

Distinct uses:
- **Constant object:** `constexpr` variable implies constant initialization.
- **Potentially constant function:** Callable in constant evaluation when arguments allow.

Example:

```
constexpr int add(int a, int b) { return a + b; }
constexpr int x = add(1, 2);
int y = add(x, 5);
```

Gotchas:
- Not every call to a `constexpr` function is a constant expression.

2.26 `constinit`

- **Meaning:** Requires that a variable with static storage duration is initialized at compile time.
- **Where it applies:** declaration (static storage duration variables).
- **Use when:**
  - You must avoid dynamic initialization order issues.
  - You want a hard guarantee of constant initialization for globals/statics.
- **Avoid when:**
  - The initializer cannot be constant-initialized.
  - You intend the variable to be immutable (use `constexpr` or `const`).
- **See also:** `constexpr`, `static`.
- **Notes (C++20):** Introduced in C++20.

Example:

```
constinit int g = 42;      // ok: constant initialization
```

Gotchas:
- `constinit` does not imply `const`.

2.27 `const_cast`

- **Meaning:** Cast that can add/remove `const`/`volatile` qualification.
- **Where it applies:** expression (cast).
- **Use when:**
  - Interfacing with legacy APIs that incorrectly lack `const`.
  - You know the original object is not actually const and you need a non-const view.
- **Avoid when:**
  - You would modify an actually-const object (undefined behavior).
  - You are using it to "work around" poor API design in new code.
- **See also:** `const`, `reinterpret_cast`.

Example:

```
void legacy(char* p);
void call(const char* p) {
  legacy(const_cast<char*>(p)); // only if legacy won't write
}
```

Gotchas:
- Writing through a pointer obtained by `const_cast` to a truly-const object is UB.

2.28 `continue`

- **Meaning:** Skips to the next iteration of the nearest enclosing loop.
- **Where it applies:** statement.
- **Use when:**
  - You want to ignore non-matching elements early in a loop.
  - It reduces nesting and improves readability.
- **Avoid when:**
  - Multiple `continue`s obscure loop invariants.
  - A clearer loop condition can eliminate the need.
- **See also:** `break`, `for`, `while`.

Example:

```
for (int i=0; i<10; ++i) {
  if (i % 2 == 1) continue;
}
```

Gotchas:
- In `do`/`while`, `continue` jumps to the condition check.

2.29 `co_await`

- **Meaning:** Suspends a coroutine until an awaitable completes and yields its result.
- **Where it applies:** expression (inside a coroutine).
- **Use when:**
  - You implement asynchronous control flow with coroutines.
  - You want to express suspension points explicitly.
- **Avoid when:**
  - You are not in a coroutine (ill-formed).
  - Simpler synchronous code is sufficient.
- **See also:** `co_return`, `co_yield`.
- **Notes (C++20):** Coroutines keywords are introduced in C++20.

Example:

```
// Illustrative only: requires a coroutine return type and promise type.
auto v = co_await something();
```

Gotchas:
- Meaning depends on the awaitable protocol; it is not a library call.

2.30 `co_return`

- **Meaning:** Returns from a coroutine, completing it (possibly producing a value).
- **Where it applies:** statement (inside a coroutine).
- **Use when:**
  - You finalize a coroutine's result.
  - You want to exit early from coroutine logic.
- **Avoid when:**
  - You are not in a coroutine.
  - You need to produce a sequence (use `co_yield`).
- **See also:** `return`, `co_yield`.

Example:

```
// Illustrative only: requires coroutine machinery.
co_return 42;
```

Gotchas:
- The value is delivered according to the promise type's rules.

2.31 `co_yield`

- **Meaning:** Produces a value from a coroutine generator and suspends.
- **Where it applies:** expression/statement (inside a coroutine).
- **Use when:**
  - You implement generators producing a sequence of values.
  - You want lazy iteration with suspension points.
- **Avoid when:**
  - A simple container/iterator is enough.
  - You are not in a coroutine.
- **See also:** `co_await`, `co_return`.

Example:

```
// Illustrative only: requires generator coroutine type.
co_yield 1;
co_yield 2;
```

Gotchas:
- Semantics depend on the promise type (e.g., yield value handling).

2.32 `decltype`

- **Meaning:** Forms a type from an expression's declared type (preserving references in specific cases).
- **Where it applies:** type; declaration; template.
- **Use when:**
  - You need the exact type of an expression in templates.
  - You want trailing return types dependent on expressions.
- **Avoid when:**
  - `auto` suffices and is clearer.
  - You are not prepared for reference-preserving rules (parentheses matter).
- **See also:** `auto`, `typename`.

Example:

```
int i = 0;
decltype(i) a = 1;     // int
decltype((i)) b = i;   // int&  (note parentheses)
```

Gotchas:
- `decltype((x))` yields an lvalue reference type when `x` is an lvalue.

2.33 `default`

- **Meaning:** Marks a `switch` default label or requests compiler-generated special member functions.
- **Where it applies:** statement; class member declaration.
- **Use when:**
  - You want explicit compiler-generated `= default` special members.
  - You want a `switch` catch-all branch.
- **Avoid when:**
  - `switch` exhaustiveness matters (omit `default` deliberately).
  - Defaulting would violate class invariants (write the function).
- **See also:** `delete`, `switch`.

Distinct uses:
- **Switch label:** `default:` as the fallback case.
- **Defaulted function:** `T() = default;` requests implicit definition.

Example:

```
struct X { X() = default; };
switch (k) { default: break; }
```

Gotchas:
- `= default` can be ill-formed if the implicit definition would be deleted.

2.34 `delete`

- **Meaning:** Deallocates dynamic storage or forbids a function via `= delete`.
- **Where it applies:** expression; declaration.
- **Use when:**
  - You release memory allocated with `new`.
  - You prevent unwanted overloads/conversions with `= delete`.
- **Avoid when:**
  - You can use automatic storage and RAII instead of manual `new`/`delete`.
  - You delete through a non-matching allocation form (must match `new`/`new[]`).
- **See also:** `new`, `default`.

Distinct uses:
- **Deallocation:** `delete p;` or `delete[] p;`
- **Deleted function:** `f(int) = delete;` forbids use.

Example:

```
struct N {
  N(int) = delete;
};
int* p = new int(1);
delete p;
```

Gotchas:
- Use `delete[]` only for memory allocated with `new[]`.

2.35 `do`

- **Meaning:** Starts a `do-while` loop (body executes at least once).
- **Where it applies:** statement.
- **Use when:**
  - You must run the loop body before checking the condition.
  - You express retry logic with a post-check.
- **Avoid when:**
  - A pre-check loop (`while`) is clearer.
  - It encourages complex control flow.
- **See also:** `while`, `for`.

Example:

```
int x = 0;
do { ++x; } while (x < 3);
```

Gotchas:
- The trailing `while(...);` semicolon is required.

2.36 `double`

- **Meaning:** Fundamental floating-point type, typically double precision.
- **Where it applies:** type.
- **Use when:**
  - You need real-number approximations with typical precision.
  - Interfacing with numeric code expecting `double`.
- **Avoid when:**
  - Exact arithmetic is required (floating-point is approximate).
  - `float` precision/space is sufficient and you have measured benefit.
- **See also:** `float`, `long`.

Example:

```
double pi = 3.141592653589793;
```

Gotchas:
- Floating-point comparisons require care with rounding error.

2.37 `dynamic_cast`

- **Meaning:** Checked cast for navigating polymorphic class hierarchies at runtime.
- **Where it applies:** expression (cast).
- **Use when:**
  - You must safely downcast across a polymorphic base.
  - You implement type-safe runtime branching when virtual dispatch is insufficient.
- **Avoid when:**
  - Virtual functions (polymorphism) can express the behavior directly.
  - The hierarchy is not polymorphic (no virtuals): cast will not work as intended.
- **See also:** `typeid`, `static_cast`.

Example:

```
struct B { virtual ~B() = default; };
struct D : B { int v = 1; };
B* b = new D{};
if (auto* d = dynamic_cast<D*>(b)) { (void)d->v; }
delete b;
```

Gotchas:
- On reference casts, failure throws; on pointer casts, failure yields `nullptr`.

2.38 `else`

- **Meaning:** Introduces the alternative branch of an `if` statement.
- **Where it applies:** statement.
- **Use when:**
  - You need mutually exclusive branches.
  - You want to express a default action when a condition is false.
- **Avoid when:**
  - Early returns make the `else` unnecessary and reduce nesting.
  - The `else` branch is empty or only comments (restructure).
- **See also:** `if`.

Example:

```
if (x > 0) { x = 1; }
else { x = -1; }
```

Gotchas:
- Beware the "dangling else"; always use braces in nested conditionals.

2.39 `enum`

- **Meaning:** Declares an enumeration type (scoped or unscoped).
- **Where it applies:** type; declaration.
- **Use when:**
  - You model a closed set of named values.
  - You want type safety with `enum class`.
- **Avoid when:**
  - Values are open-ended or require validation (consider stronger types).
  - You rely on implicit conversions (prefer scoped enums).
- **See also:** `class`, `switch`.

Example:

```
enum class Color { Red, Green, Blue };
```

Gotchas:
- Unscoped `enum` injects enumerator names into the enclosing scope.

2.40 `explicit`

- **Meaning:** Prevents implicit conversions for constructors and conversion functions.
- **Where it applies:** constructor; conversion function.
- **Use when:**
  - A single-argument constructor should not enable implicit conversions.
  - You define a conversion operator that should require a cast/context.
- **Avoid when:**
  - A safe, obvious implicit conversion improves usability and cannot surprise.
  - You are mirroring a built-in type's implicit conversion behavior deliberately.

- **See also:** `operator`.

Distinct uses:
- **Explicit constructor:** blocks implicit construction from argument.
- **Explicit conversion operator:** requires explicit cast/context.

Example:

```
struct S {
  explicit S(int) {}
  explicit operator bool() const { return true; }
};
```

Gotchas:
- `explicit` does not affect direct-initialization (e.g., `S s(1);`).

2.41 `export`

- **Meaning:** Exports declarations from a module interface unit.
- **Where it applies:** module interface; declaration.
- **Use when:**
  - You author a C++20 module interface and want names visible to importers.
  - You want to hide internal implementation details in module partitions/impl units.
- **Avoid when:**
  - You are not using modules (it is then meaningless/ill-formed).
  - You export everything by default (export only the intended interface).
- **See also:** `module`, `import`.
- **Notes (C++20):** `export` is used for modules; the old templates-related `export` was removed long ago and is not part of C++20 templates.

Example:

```
// module interface unit:
export module M;
export int f();
```

Gotchas:
- Module support is compiler/toolchain dependent even in C++20 mode.

2.42 `extern`

- **Meaning:** Specifies linkage or declares an entity defined elsewhere.
- **Where it applies:** declaration; linkage specification.
- **Use when:**
  - You declare a variable/function with external linkage.
  - You use `extern "C"` to control language linkage for interop.
- **Avoid when:**
  - You create global variables unnecessarily (prefer encapsulation).
  - You hide ODR issues with scattered `extern` declarations.
- **See also:** `static`, `namespace`.

Example:

```
extern int g;
extern "C" void c_api();
```

Gotchas:
- `extern` does not allocate storage unless it is a definition with initializer (special cases exist).

2.43 `false`

- **Meaning:** Boolean literal with value `false`.
- **Where it applies:** expression.
- **Use when:**
  - You need a boolean constant in code or templates.
  - You initialize boolean state explicitly.
- **Avoid when:**
  - A named constant would better document intent.
  - Using `false` as a sentinel for multiple meanings (prefer `enum`).
- **See also:** `true`, `bool`.

Example:

```
bool ok = false;
```

Gotchas:
- Prefer clear naming rather than relying on `true`/`false` positional meaning.

2.44 `float`

- **Meaning:** Fundamental floating-point type, typically single precision.
- **Where it applies:** type.
- **Use when:**
  - You need lower precision and memory/bandwidth matter.
  - You interface with APIs or data formats that use `float`.
- **Avoid when:**
  - Precision is important and you have not measured error bounds.
  - You perform iterative accumulation sensitive to rounding.
- **See also:** `double`.

Example:

```
float x = 1.0f;
```

Gotchas:
- Literals default to `double`; use `f` suffix for `float`.

2.45 `for`

- **Meaning:** Introduces a `for` loop (including range-based form).
- **Where it applies:** statement.
- **Use when:**
  - You have a clear iteration pattern with init/condition/increment.
  - Range-based `for` expresses traversal directly.
- **Avoid when:**
  - The loop logic is unclear (prefer algorithms or clearer control flow).
  - Complex increment/condition turns it into a "mini program".
- **See also:** `while`, `break`, `continue`.

Example:

```
for (int i = 0; i < 3; ++i) { /*...*/ }
```

Gotchas:
- Prefer braces even for single-statement bodies to avoid errors.

2.46 `friend`

- **Meaning:** Grants access to private/protected members to a specified function or class.
- **Where it applies:** class member declaration.
- **Use when:**
  - A non-member function requires intimate access for symmetry (e.g., operators).
  - A tightly coupled helper needs access without expanding the public API.
- **Avoid when:**
  - It indicates poor encapsulation boundaries.
  - It creates broad privileged access across modules.
- **See also:** `private`, `class`.

Example:

```
class X {
  int v = 0;
  friend int get(const X& x) { return x.v; }
};
```

Gotchas:
- Friendship is not transitive and not inherited.

2.47 `goto`

- **Meaning:** Jumps unconditionally to a labeled statement within a function.
- **Where it applies:** statement.
- **Use when:**
  - Only in rare low-level patterns (e.g., breaking out of deeply nested logic) with clear style rules.
- **Avoid when:**
  - Structured control flow can express the logic.
  - It would skip initialization of variables with automatic storage (ill-formed in many cases).
- **See also:** `break`, `return`.

Example:

```
int f(bool ok) {
  if (!ok) goto fail;
  return 0;
fail:
  return -1;
}
```

Gotchas:
- Jumping past initialization of non-trivial objects is ill-formed.

2.48 `if`

- **Meaning:** Conditionally executes a statement based on a boolean condition.
- **Where it applies:** statement.
- **Use when:**
  - You branch on a condition (possibly with init-statement in C++17+).
  - You validate preconditions and return/throw early.
- **Avoid when:**
  - Multi-way discrete dispatch fits `switch` better.
  - Deeply nested `if` chains hide invariants (refactor).
- **See also:** `else`, `switch`.

Example:

```
if (int x = 1; x > 0) { (void)x; }
```

Gotchas:
- Always use braces in nested conditionals to avoid dangling-`else` mistakes.

2.49 `import`

- **Meaning:** Imports a module unit into the current translation unit (contextual keyword).
- **Where it applies:** module; declaration region.
- **Use when:**
  - You consume C++20 modules instead of textual includes.
  - You want cleaner separation of interface and implementation.
- **Avoid when:**
  - Your toolchain/build system cannot reliably support modules yet.
  - You mix `import` and fragile include-order dependencies.
- **See also:** `module`, `export`.
- **Notes (C++20):** `import` is a contextual keyword in module contexts.

Example:

```
import M;
```

Gotchas:
- `import` being contextual means it is not always treated as a keyword outside module syntax.

2.50 `inline`

- **Meaning:** Allows multiple definitions across translation units (ODR) for certain entities; suggests inlining.
- **Where it applies:** function; variable (C++17+); namespace.
- **Use when:**
  - You define small functions in headers with ODR-safe semantics.
  - You define header-visible variables as `inline` to avoid multiple definitions.
- **Avoid when:**
  - You use it as a performance directive (the optimizer decides).
  - It hides large function bodies in headers unnecessarily.
- **See also:** `static`, `extern`.

Example:

```
inline int f(int x) { return x + 1; }
```

Gotchas:
- `inline` affects linkage/ODR; it does not guarantee inlining.

2.51 `int`

- **Meaning:** Fundamental signed integer type.
- **Where it applies:** type.
- **Use when:**
  - You need a general-purpose integer for counts/indices within range.
  - You interface with APIs that use `int`.
- **Avoid when:**
  - Width must be fixed (use fixed-width types in the standard library, not covered here).
  - You need non-negative semantics (consider unsigned carefully).
- **See also:** `short`, `long`, `signed`, `unsigned`.

Example:

```
int n = 42;
```

Gotchas:
- Signed overflow is undefined behavior; avoid relying on it.

2.52 `long`

- **Meaning:** Integer or floating modifier for a wider type (e.g., `long int`, `long double`).
- **Where it applies:** type.
- **Use when:**
  - You want a wider integer than `int` where available.
  - You explicitly need `long double` precision.
- **Avoid when:**
  - You assume exact bit-width portability.
  - You mix types without explicit conversion rules.
- **See also:** `short`, `double`.

Example:

```
long n = 0;
long double x = 0.0L;
```

Gotchas:
- Sizes of `long` and `long double` are implementation-defined.

2.53 `module`

- **Meaning:** Declares a module unit or module interface/partition (contextual keyword).
- **Where it applies:** translation unit (module declaration).
- **Use when:**
  - You define modular interfaces with explicit exports.
  - You want faster builds and cleaner dependency graphs.
- **Avoid when:**
  - Your toolchain does not support modules adequately.
  - You treat modules like headers without designing an explicit interface.
- **See also:** `export`, `import`.
- **Notes (C++20):** `module` is contextual within module syntax.

Example:

```
export module M;
```

Gotchas:
- Module partitions and global module fragments have special ordering rules.

2.54 `mutable`

- **Meaning:** Allows a data member to be modified even in a `const` object.
- **Where it applies:** class member declaration.
- **Use when:**
  - You implement caching/memoization that does not change logical state.
  - You store synchronization primitives used by `const` operations.
- **Avoid when:**
  - It is used to bypass a poorly designed `const` interface.
  - It makes "const" meaningless by mutating true state.
- **See also:** `const`.

Example:

```
struct S {
  mutable int cache = 0;
  int get() const { return ++cache; } // logical constness assumed
};
```

Gotchas:
- `mutable` can silently break thread-safety assumptions.

2.55 `namespace`

- **Meaning:** Declares a namespace scope for grouping names.
- **Where it applies:** declaration; scope.
- **Use when:**
  - You prevent name collisions and structure APIs.
  - You create internal implementation namespaces.
- **Avoid when:**
  - You put everything in the global namespace.
  - You use deep nesting without benefit (keep it purposeful).
- **See also:** `using`, `extern`.

Example:

```
namespace app { int f(); }
```

Gotchas:
- Avoid `using namespace ...;` at global scope in headers.

2.56 `new`

- **Meaning:** Allocates dynamic storage and constructs an object.
- **Where it applies:** expression.
- **Use when:**
  - You must manage lifetime dynamically and cannot use automatic storage.
  - You implement low-level allocation strategies behind safe APIs.
- **Avoid when:**
  - RAII and automatic storage can express ownership more safely.
  - You pair it with manual `delete` in error-prone code paths.
- **See also:** `delete`, `throw`.

Example:

```
int* p = new int(7);
delete p;
```

Gotchas:
- Exceptions between `new` and `delete` can leak without RAII.

2.57 `noexcept`

- **Meaning:** Specifies that a function does not throw exceptions (or forms a boolean operator).
- **Where it applies:** function; expression.
- **Use when:**
  - You can guarantee no exceptions escape (especially move operations).
  - You want stronger guarantees for callers and optimizations.
  - You query whether an expression is declared non-throwing via `noexcept(expr)`.
- **Avoid when:**
  - The function may throw; incorrect `noexcept` causes termination.
  - You use it as decoration without proof.
- **See also:** `throw`, `try`.

Distinct uses:
- **Noexcept specifier:** `f() noexcept` promises no exception escapes.
- **Noexcept operator:** `noexcept(expr)` yields `true`/`false` at compile time.

Example:

```
void f() noexcept {}
static_assert(noexcept(f()));
```

Gotchas:
- Violating `noexcept` typically calls `std::terminate()` (conceptually; behavior is terminate).

2.58 `not`

- **Meaning:** Alternative token for `!` (logical NOT).
- **Where it applies:** expression.
- **Use when:**
  - Word operators improve readability in boolean expressions.
  - You want to avoid punctuation-heavy conditions.
- **Avoid when:**
  - Codebase standard uses `!`.
  - Mixing styles hurts consistency.
- **See also:** `and`, `or`.

Example:

```
if (not ok) { /*...*/ }
```

Gotchas:
- Same precedence and semantics as `!`.

2.59 `not_eq`

- **Meaning:** Alternative token for `!=`.
- **Where it applies:** expression.
- **Use when:**
  - You prefer word operators in comparisons for readability.
- **Avoid when:**
  - Your codebase uses `!=` consistently.
  - It surprises readers unfamiliar with alternative tokens.
- **See also:** `not`, `or`.

Example:

```
if (a not_eq b) { /*...*/ }
```

Gotchas:
- This is a token alias; it does not change overload resolution rules.

2.60 `nullptr`

- **Meaning:** A null pointer literal of type `std::nullptr_t` (keyword literal).
- **Where it applies:** expression.
- **Use when:**
  - You represent "no object" for pointers.
  - You want unambiguous overload resolution vs integer `0`.
- **Avoid when:**
  - You use sentinel pointers where optional semantics should be explicit.
  - You compare to `0` or `NULL` (prefer `nullptr`).
- **See also:** `delete`, `new`.

Example:

```
int* p = nullptr;
if (p == nullptr) { /*...*/ }
```

Gotchas:
- `nullptr` converts to any pointer type, but not to integral types.

2.61 `operator`

- **Meaning:** Declares an overloaded operator function or a conversion function.
- **Where it applies:** function; class member; conversion.
- **Use when:**
  - You provide natural syntax consistent with the type's semantics.
  - You implement comparison/arithmetic as conventional operations.
- **Avoid when:**
  - The operator meaning would be surprising or non-obvious.
  - You overload operators purely for cleverness or side effects.
- **See also:** `explicit`, `friend`.

Example:

```
struct X {
  int v;
  X operator+(const X& o) const { return {v + o.v}; }
};
```

Gotchas:
- Some operators have fixed arity/meaning constraints (cannot invent new operators).

2.62 `or`

- **Meaning:** Alternative token for `||` (logical OR).
- **Where it applies:** expression.
- **Use when:**
  - Word operators improve readability for boolean logic.
  - You want consistency with `and`/`not`.
- **Avoid when:**
  - Codebase standardizes on `||`.
  - It could be confused with bitwise `bitor`.
- **See also:** `and`, `not`, `bitor`.

Example:

```
bool ok = fast or safe;
```

Gotchas:
- Short-circuit semantics are identical to `||`.

2.63 `or_eq`

- **Meaning:** Alternative token for `|=` (bitwise OR assignment).
- **Where it applies:** expression.
- **Use when:**
  - You prefer word operators for bitmask updates.
- **Avoid when:**
  - Your codebase uses `|=` consistently.
  - It could be confused with logical `or`.
- **See also:** `bitor`, `and_eq`.

Example:

```
unsigned f = 0;
f or_eq 0b0010u;
```

Gotchas:
- Bitwise operation; not a boolean operation.

2.64 `private`

- **Meaning:** Restricts member access to the class and its friends.
- **Where it applies:** class member access specifier.
- **Use when:**
  - You hide representation and enforce invariants.
  - You prevent clients from depending on internals.
- **Avoid when:**
  - It blocks legitimate extension points (consider `protected` or public API).
  - You fight the design by adding too many friends.
- **See also:** `public`, `protected`, `friend`.

Example:

```
class X {
private:
  int v = 0;
};
```

Gotchas:
- Access control affects name lookup and overload resolution subtleties.

2.65 `protected`

- **Meaning:** Restricts member access to the class, its friends, and derived classes.
- **Where it applies:** class member access specifier.
- **Use when:**
  - You design for inheritance and need controlled extension points.
  - Derived classes must access internals to implement customization.
- **Avoid when:**
  - Inheritance is not intended (prefer `private` + public/protected functions).
  - It exposes too much representation to derived classes (fragile base class).
- **See also:** `private`, `public`, `virtual`.

Example:

```
struct B { protected: int state = 0; };
```

Gotchas:
- `protected` data members are often a design smell; prefer protected functions.

2.66 `public`

- **Meaning:** Makes class members accessible to all callers.
- **Where it applies:** class member access specifier.
- **Use when:**
  - You declare the intended interface of a type.
  - You expose non-member-like operations as member functions.
- **Avoid when:**
  - You expose representation (public data) that must preserve invariants.
  - You publish members you may need to change without breaking users.
- **See also:** `private`, `protected`.

Example:

```
struct S { public: int x = 0; };
```

Gotchas:
- In `struct`, members are `public` by default; in `class`, `private`.

2.67 `register`

- **Meaning:** Historical storage-class specifier suggesting register storage.
- **Where it applies:** declaration.
- **Use when:**
  - Never in modern C++.
- **Avoid when:**
  - Always; it is deprecated and provides no benefit.
- **Notes (C++20):** `register` is deprecated; compilers ignore it.

Example:

```
// Deprecated: register int x = 0;
int x = 0;
```

Gotchas:
- Deprecated entities may trigger warnings or be removed in future standards.

2.68 `reinterpret_cast`

- **Meaning:** Performs low-level, implementation-defined reinterpretation casts.
- **Where it applies:** expression (cast).
- **Use when:**
  - You implement low-level interop/ABI boundaries with careful constraints.
  - You need to cast between unrelated pointer types in controlled code (with documented invariants).
- **Avoid when:**
  - You want numeric conversion or safe downcasts (use other casts).
  - It would violate strict aliasing or object lifetime rules.
- **See also:** `static_cast`, `const_cast`.

Example:

```
void* p = nullptr;
auto* q = reinterpret_cast<char*>(p);
(void)q;
```

Gotchas:
- Many uses lead to undefined behavior if you access objects through invalid types.

2.69 `requires`

- **Meaning:** Introduces template constraints or a requires-expression.
- **Where it applies:** template; declaration; expression.
- **Use when:**
  - You constrain templates for clearer intent and diagnostics.
  - You express ad-hoc requirements in a `requires` expression.
- **Avoid when:**
  - Constraints are duplicated instead of refactored into a `concept`.
  - You over-constrain and reduce generic usability.
- **See also:** `concept`, `template`.
- **Notes (C++20):** `requires` is introduced in C++20.

Distinct uses:
- **Requires clause:** constrains a template or function.
- **Requires expression:** checks validity of expressions/types.

Example:

```
template<class T>
requires requires(T x) { ++x; }
T inc(T x) { return ++x; }
```

Gotchas:
- Constraints participate in overload resolution; small differences can change selected overloads.

2.70 `return`

- **Meaning:** Exits a function, optionally providing a value.
- **Where it applies:** statement.
- **Use when:**
  - You produce a function result or terminate early.
  - You reduce nesting via guard clauses.
- **Avoid when:**
  - Many scattered returns obscure cleanup (prefer RAII instead of manual cleanup).
  - You return references/pointers to dead objects (lifetime errors).
- **See also:** `co_return`, `throw`.

Example:

```
int f(int x) {
  if (x < 0) return -1;
  return x + 1;
}
```

Gotchas:
- Returning a reference requires that the referred object outlives the caller.

2.71 `short`

- **Meaning:** Integer modifier for a type no wider than `int` (e.g., `short int`).
- **Where it applies:** type.
- **Use when:**
  - You need a potentially smaller integer and accept implementation-defined width.
  - Interfacing with ABIs that require `short`.
- **Avoid when:**
  - You assume exact width/portability.
  - `int` is fine and clearer for arithmetic.
- **See also:** `int`, `long`.

Example:

```
short s = 1;
```

Gotchas:
- Usual arithmetic promotions may convert `short` to `int` in expressions.

2.72 `signed`

- **Meaning:** Specifies a signed integer type (often redundant for `int`).
- **Where it applies:** type.
- **Use when:**
  - You need to disambiguate `signed char` vs `char`.
  - You document signedness intentionally in low-level code.
- **Avoid when:**
  - It adds noise without increasing clarity (e.g., `signed int`).
  - Unsigned is semantically required (counts/bitmasks) and you mix them carelessly.
- **See also:** `unsigned`, `char`.

Example:

```
signed char sc = -1;
```

Gotchas:
- `char` is distinct from `signed char` and may have different signedness.

2.73 `sizeof`

- **Meaning:** Yields the size in bytes of a type or object representation.
- **Where it applies:** expression; type.
- **Use when:**
  - You need compile-time size information (arrays, layouts).
  - You implement low-level serialization with explicit constraints.
- **Avoid when:**
  - You use it to compute element counts from pointers (loss of array size).
  - You assume layout stability across compilers/ABIs without guarantees.
- **See also:** `alignof`, `decltype`.

Example:

```
int a[3]{};
static_assert(sizeof(a) == 3 * sizeof(int));
```

Gotchas:
- `sizeof` does not evaluate its operand (except in some cases with VLA-like extensions).

2.74 `static`

- **Meaning:** Specifies static storage duration or class-level membership, depending on context.
- **Where it applies:** declaration; class member.
- **Use when:**
  - You need a function-local static with persistent state (with care).
  - You define class members shared across instances.
  - You give internal linkage to a namespace-scope entity (though unnamed namespaces often preferred).
- **Avoid when:**
  - You create hidden global state that complicates testing and reasoning.
  - You use it in headers without `inline` and cause ODR/linkage surprises.
- **See also:** `thread_local`, `inline`, `extern`.

Distinct uses:
- **Static storage duration:** `static int x;` at namespace/function scope.
- **Static class member:** shared across all objects.

Example:

```
struct X { static int count; };
int X::count = 0;
```

Gotchas:
- Initialization order of non-local statics across translation units can be problematic.

2.75 `static_assert`

- **Meaning:** Performs a compile-time assertion with an optional message.
- **Where it applies:** declaration; statement.
- **Use when:**
  - You enforce invariants about types, sizes, or constraints at compile time.
  - You document assumptions that must hold on all targets.
- **Avoid when:**
  - The condition depends on runtime data.
  - The assertion duplicates what the type system already guarantees.
- **See also:** `constexpr`, `requires`.

Example:

```
static_assert(sizeof(int) >= 4, "int too small");
```

Gotchas:
- Keep messages stable and actionable for maintainers.

2.76 `static_cast`

- **Meaning:** Performs a compile-time checked cast within well-defined conversion categories.
- **Where it applies:** expression (cast).
- **Use when:**
  - You need an explicit numeric conversion.
  - You upcast/downcast in non-polymorphic or known-safe situations.
- **Avoid when:**
  - You need runtime-checked downcast (use `dynamic_cast`).
  - You are reinterpreting representation (do not use as `reinterpret_cast`).
- **See also:** `dynamic_cast`, `reinterpret_cast`, `const_cast`.

Example:

```
int x = 1;
double y = static_cast<double>(x);
```

Gotchas:
- A `static_cast` can still produce narrowing or lossy conversions; it just makes it explicit.

2.77 `struct`

- **Meaning:** Declares a class type; default member access is `public`.
- **Where it applies:** class; type; declaration.
- **Use when:**
  - You model a passive aggregate-like data carrier by convention.
  - You want public-by-default members for simple types.
- **Avoid when:**
  - You need strict encapsulation and invariants (prefer `class`).
  - Public data will lead to invalid states.
- **See also:** `class`, `public`, `private`.

Example:

```
struct Point { int x=0, y=0; };
```

Gotchas:
- Default inheritance access is `public` for `struct`.

2.78 `switch`

- **Meaning:** Multi-way branch based on an integral or enumeration value.
- **Where it applies:** statement.
- **Use when:**
  - You dispatch on discrete values with many cases.
  - You want to centralize branching logic in a single construct.
- **Avoid when:**
  - Conditions are not based on discrete integral/enum values.
  - Fallthrough would be subtle or unsafe.
- **See also:** `case`, `default`, `break`.

Example:

```
switch (n) {
  case 1: return 10;
  default: return 0;
}
```

Gotchas:
- Always document intentional fallthrough (use braces and comments).

2.79 `synchronized`

- **Meaning:** Reserved keyword related to transactional memory (TM) synchronization.
- **Where it applies:** (reserved; not used in standard portable C++20).
- **Use when:**
  - Only with specific compiler extensions that define it.
- **Avoid when:**
  - Writing portable C++20 code.
  - Using it as an identifier.
- **Notes (C++20):** Reserved for potential TM-related use; not a portable concurrency primitive.

Example:

```
// Reserved keyword; no standard portable usage.
```

Gotchas:
- Do not confuse with library-level synchronization primitives (not keywords).

2.80 `template`

- **Meaning:** Introduces a template declaration or disambiguates dependent names.
- **Where it applies:** template; declaration; expression (dependent context).
- **Use when:**
  - You define generic code parameterized by types/values/templates.
  - You disambiguate a dependent member template call.
- **Avoid when:**
  - You can express the abstraction without templates (simpler code).
  - You write unconstrained templates that accept too much (use `requires`/`concept`).
- **See also:** `typename`, `requires`, `concept`.

Distinct uses:
- **Template declaration:** `template<...>` introduces template parameters.
- **Dependent template disambiguator:** `x.template f<int>()`.

Example:

```
template<class T>
struct Box { T v; };

template<class T>
void g(T& x) { (void)x.template operator()<int>(); } // illustrative
```

Gotchas:
- The disambiguator `template` is required only in dependent contexts.

2.81 `this`

- **Meaning:** Pointer to the current object within a non-static member function.
- **Where it applies:** expression (member function body).
- **Use when:**
  - You need to refer to the current object explicitly.
  - You return `*this` for fluent interfaces (with care).
- **Avoid when:**
  - It reduces readability (prefer direct member access).
  - You expose raw pointer semantics unnecessarily.
- **See also:** `const`, `return`.

Example:

```
struct S {
  int x = 0;
  S& set(int v) { this->x = v; return *this; }
};
```

Gotchas:
- In `const` member functions, `this` points to `const` object type.

2.82 `thread_local`

- **Meaning:** Specifies thread storage duration for an object (one instance per thread).
- **Where it applies:** declaration.
- **Use when:**
  - You need per-thread caches/state without shared synchronization.
  - You implement thread-local performance optimizations carefully.
- **Avoid when:**
  - You need explicit lifetime control (thread exit timing matters).
  - It complicates testing or introduces hidden state.
- **See also:** `static`.

Example:

```
thread_local int tls_counter = 0;
```

Gotchas:
- Initialization/destruction timing is tied to thread lifetime, not scope.

2.83 `throw`

- **Meaning:** Throws an exception or rethrows the current exception.
- **Where it applies:** expression/statement.
- **Use when:**
  - You signal an error that cannot be handled locally.
  - You rethrow inside a `catch` to propagate.
- **Avoid when:**
  - Control flow is normal/expected (exceptions are for exceptional cases).
  - You throw from destructors during stack unwinding (leads to termination).
- **See also:** `try`, `catch`, `noexcept`.

Example:

```
try { throw 1; }
catch (...) { throw; } // rethrow
```

Gotchas:
- Throwing during stack unwinding can call termination if another exception is active.

2.84 `true`

- **Meaning:** Boolean literal with value `true`.
- **Where it applies:** expression.
- **Use when:**
  - You express a boolean constant directly.
  - You set default boolean state explicitly.
- **Avoid when:**
  - A named constant would convey meaning better.
  - It obscures intent in function calls (consider named parameters pattern).
- **See also:** `false`, `bool`.

Example:

```
bool enabled = true;
```

Gotchas:
- Positional booleans in APIs often reduce readability.

2.85 `try`

- **Meaning:** Begins a block that is monitored for exceptions to be handled by `catch`.
- **Where it applies:** statement; function-try-block; constructor.
- **Use when:**
  - You can handle exceptions locally and maintain invariants.
  - You need a function-try-block for constructors to catch initialization failures.
- **Avoid when:**
  - You catch merely to log and continue incorrectly.
  - Broad try-blocks hide which operation can throw.
- **See also:** `catch`, `throw`.

Example:

```
try { /* work */ }
catch (...) { /* handle */ }
```

Gotchas:
- Catching everything can swallow important error information if mishandled.

2.86 `typedef`

- **Meaning:** Creates a type alias (older syntax).
- **Where it applies:** declaration.
- **Use when:**
  - You maintain legacy code using `typedef`.
  - You need a quick alias in older-style declarations.
- **Avoid when:**
  - New code can use `using` aliases (usually clearer).
  - The alias hides important type information in public APIs.
- **See also:** `using`.

Example:

```
typedef int i32;
```

Gotchas:
- `typedef` cannot express template aliases; `using` can.

2.87 `typeid`

- **Meaning:** Produces runtime type information for an expression or type.
- **Where it applies:** expression; type.
- **Use when:**
  - You need RTTI for diagnostics/logging in polymorphic code.
  - You implement limited runtime type checks where design requires it.
- **Avoid when:**
  - Virtual dispatch expresses behavior more cleanly.
  - You use it for control flow instead of proper polymorphism.
- **See also:** `dynamic_cast`.

Example:

```
struct B { virtual ~B() = default; };
B* b = nullptr;
(void)typeid(*b); // may throw if b is null and type is polymorphic
```

Gotchas:
- `typeid(*p)` on a null pointer to polymorphic type can throw.

2.88 `typename`

- **Meaning:** Marks a dependent name as a type in templates.
- **Where it applies:** template; type; declaration.
- **Use when:**
  - You refer to a dependent nested type (e.g., `T::value_type`).
  - You write template code that must disambiguate types vs values.
- **Avoid when:**
  - The name is non-dependent (unnecessary noise).
  - You use it incorrectly where a value is expected.
- **See also:** `template`, `decltype`.

Distinct uses:
- **Dependent type disambiguator:** `typename T::X`.
- **Template parameter:** `template<typename T>` as a synonym for `class`.

Example:

```
template<typename T>
void f() {
  typename T::value_type* p = nullptr;
  (void)p;
}
```

Gotchas:
- Missing `typename` in dependent contexts is a common template error.

2.89 `union`

- **Meaning:** Declares a union type whose members share storage.
- **Where it applies:** type; declaration.
- **Use when:**
  - You implement low-level tagged unions with explicit active-member tracking.
  - You need a layout overlay for ABI/serialization with strict rules.
- **Avoid when:**
  - A safer discriminated union abstraction is appropriate (not covered here).
  - Active member lifetime rules are not carefully managed.
- **See also:** `struct`, `reinterpret_cast`.

Example:

```
union U { int i; double d; };
U u{}; u.i = 1;
```

Gotchas:
- Reading an inactive member can violate lifetime/aliasing rules.

2.90 `unsigned`

- **Meaning:** Specifies an unsigned integer type.
- **Where it applies:** type.
- **Use when:**
  - You model bitmasks and modular arithmetic intentionally.
  - You need well-defined wraparound behavior.
- **Avoid when:**
  - You use it for "must be non-negative" in general arithmetic (mixing signed/unsigned hazards).
  - Comparisons with signed values can cause surprising conversions.
- **See also:** `signed`, `int`.

Example:

```
unsigned m = 0b1010u;
```

Gotchas:
- Signed/unsigned mixing can change comparison results due to usual conversions.

2.91 `using`

- **Meaning:** Introduces a using-declaration/directive or defines a type alias.
- **Where it applies:** declaration; namespace; type.
- **Use when:**
  - You create readable type aliases with `using`.
  - You bring specific names into scope with a using-declaration.
  - You simplify inheritance overload sets via `using Base::f;`.
- **Avoid when:**
  - You use `using namespace ...;` at global scope in headers.
  - Aliases hide important semantics in public APIs.
- **See also:** `typedef`, `namespace`.

Distinct uses:
- **Type alias:** `using Name = T;`.
- **Using declaration:** `using N::x;` imports one name.
- **Using directive:** `using namespace N;` imports all names (discouraged in headers).

Example:

```
namespace N { int x = 0; }
using N::x;
using I = int;
```

Gotchas:
- Using directives can cause surprising name clashes and ADL effects.

2.92 `virtual`

- **Meaning:** Declares a member function as dynamically dispatched or marks virtual inheritance.
- **Where it applies:** class member function; base class specifier.
- **Use when:**
  - You design runtime polymorphism via interfaces/base classes.
  - You need virtual destructors for polymorphic deletion.
- **Avoid when:**
  - Static polymorphism or composition better matches the design.
  - You add virtuals to concrete types without a stable base interface.
- **See also:** `override` (contextual identifier, not a keyword), `dynamic_cast`.

Example:

```
struct B { virtual ~B() = default; virtual int f() const { return 0; } };
struct D : B { int f() const { return 1; } };
```

Gotchas:
- Deleting through a base pointer requires a virtual destructor.

2.93 `void`

- **Meaning:** Indicates absence of a value/type or an untyped pointer `void*`.
- **Where it applies:** type; function return type; pointer type.
- **Use when:**
  - A function does not return a value.
  - You pass raw memory across a C ABI boundary (`void*`) with strict contracts.
- **Avoid when:**
  - You can use typed interfaces and avoid untyped pointers.
  - You use `void*` in C++ APIs without clear ownership and lifetime rules.

- **See also:** `decltype`, `reinterpret_cast`.

Example:

```
void f() {}
void* p = nullptr;
```

Gotchas:
- Converting `void*` to typed pointers requires explicit casts in C++.

2.94 `volatile`

- **Meaning:** Qualifies an object as having externally observable changes (special access semantics).
- **Where it applies:** type; declaration.
- **Use when:**
  - Interfacing with memory-mapped I/O or signal/interrupt-updated data under platform rules.
  - You must prevent certain optimizations for hardware interaction.
- **Avoid when:**
  - You try to implement thread synchronization (volatile is not a concurrency primitive).
  - You apply it broadly "just in case".
- **See also:** `const`.

Example:

```
volatile int* mmio = nullptr; // illustrative only
(void)mmio;
```

Gotchas:
- Volatile does not make operations atomic and does not establish inter-thread ordering.

2.95 `wchar_t`

- **Meaning:** Fundamental wide character type with implementation-defined encoding/size.
- **Where it applies:** type.
- **Use when:**
  - You must interoperate with platform APIs using `wchar_t`.
- **Avoid when:**
  - You assume a specific Unicode encoding (it is platform-dependent).
  - You can use explicit UTF-typed code units (`char16_t`/`char32_t`/`char8_t`) instead.
- **See also:** `char16_t`, `char32_t`, `char8_t`.

Example:

```
wchar_t wc = L'A';
```

Gotchas:
- Width/encoding differs across Windows and Unix-like platforms.

2.96 `while`

- **Meaning:** Introduces a pre-check loop that repeats while a condition is true.
- **Where it applies:** statement.
- **Use when:**
  - The number of iterations is not known up front.
  - You want a simple loop with a clear continuation condition.
- **Avoid when:**
  - A `for` loop expresses the iteration structure better.
  - Complex loop bodies obscure the condition (refactor).
- **See also:** `do`, `for`.

Example:

```
int x = 0;
while (x < 3) { ++x; }
```

Gotchas:
- Beware accidental infinite loops; keep the update visible and correct.

2.97 `xor`

- **Meaning:** Alternative token for `^` (bitwise XOR).
- **Where it applies:** expression.
- **Use when:**
  - Word form improves readability for bitwise XOR.
- **Avoid when:**
  - Your codebase standard uses `^`.
  - Readers may mistake it for logical exclusive-or (still bitwise).
- **See also:** `xor_eq`, `bitand`.

Example:

```
unsigned m = 0b1100u xor 0b1010u; // 0b0110
```

Gotchas:
- Bitwise XOR is not the same as inequality on booleans for all types.

2.98 `xor_eq`

- **Meaning:** Alternative token for `^=` (bitwise XOR assignment).
- **Where it applies:** expression.
- **Use when:**
  - You want word form in bitmask update code.
- **Avoid when:**
  - Your codebase standard uses `^=`.
  - It reduces familiarity for readers.
- **See also:** `xor`, `and_eq`.

Example:

```
unsigned f = 0b0101u;
f xor_eq 0b0011u; // 0b0110
```

Gotchas:
- Still bitwise; ensure operand types are intended.

3 Quick Reference Tables

Category	Keywords
Declarations / Specifiers	`alignas`, `alignof`, `asm`, `auto`, `const`, `consteval`, `constexpr`, `constinit`, `explicit`, `extern`, `inline`, `mutable`, `noexcept`, `register`, `static`, `thread_local`, `typedef`, `using`, `volatile`
Control flow	`break`, `case`, `continue`, `default`, `do`, `else`, `for`, `goto`, `if`, `return`, `switch`, `while`
Types / Casts	`bool`, `char`, `char8_t`, `char16_t`, `char32_t`, `double`, `float`, `int`, `long`, `short`, `signed`, `unsigned`, `void`, `wchar_t`, `const_cast`, `dynamic_cast`, `reinterpret_cast`, `static_cast`, `decltype`, `typeid`
Templates	`concept`, `requires`, `template`, `typename`
Exception handling	`try`, `catch`, `throw`, `noexcept`
Concurrency	`thread_local` (core), reserved TM: `atomic_cancel`, `atomic_commit`, `atomic_noexcept`, `synchronized`
Coroutines	`co_await`, `co_return`, `co_yield`
Other	`class`, `struct`, `union`, `enum`, `friend`, `this`, `new`, `delete`, `nullptr`, `operator`, alt tokens: `and`, `and_eq`, `bitand`, `bitor`, `compl`, `not`, `not_eq`, `or`, `or_eq`, `xor`, `xor_eq`, modules: `module`, `import`, `export`, literals: `true`, `false`

4 Index

Index

alignas, 1
alignof, 1
and, 2
and_eq, 2
asm, 2
atomic_cancel, 3
atomic_commit, 3
atomic_noexcept, 3
auto, 1, 4
    Deduced return type, 4
    Generic lambda, 4
    Variable deduction, 4

bitand, 4
bitor, 4
bool, 5
break, 1, 5

case, 5
catch, 6
char, 6
char16_t, 7
char32_t, 7
char8_t, 6
class, 1, 7
co_await, 11
co_return, 11
co_yield, 11
compl, 8
concept, 1, 8
const, 1, 8
    member function, 1, 8
    object, 8
    pointee, 8
    pointer, 8
const_cast, 10
consteval, 1, 9
constexpr, 1, 9
    function, 9
    variable, 9
constinit, 10
continue, 1, 10

decltype, 12
default, 1, 12
    defaulted function, 12
    switch, 12
delete, 12
    deallocation, 12
    deleted function, 12
do, 13
double, 13
dynamic_cast, 13

else, 14
enum, 1, 14
explicit, 14
    constructor, 15
    conversion operator, 15
export, 15
extern, 15

false, 16
float, 16
for, 1, 16
friend, 17

goto, 1, 17

if, 1, 17
import, 18
inline, 18
int, 18

long, 19

module, 19
mutable, 1, 19

namespace, 20
new, 20
noexcept, 1, 20
    operator, 20
    specifier, 20
not, 21
not_eq, 21
nullptr, 21

operator, 22
or, 22
or_eq, 22

private, 23
protected, 23
public, 23

register, 1, 24
reinterpret_cast, 1, 24
requires, 1, 24
    clause, 24
    expression, 24
return, 25

short, 25
signed, 25
sizeof, 26
static, 26
    class member, 26
    storage duration, 26

static_assert, 26
static_cast, 1, 27
struct, 27
switch, 1, 27
synchronized, 28

template, 28
    declaration, 28
    disambiguator, 28
this, 28
thread_local, 29
throw, 29
true, 29
try, 30
typedef, 30
typeid, 30
typename, 31
    dependent type, 31
    template parameter, 31

union, 31
unsigned, 31
using, 32
    declaration, 32
    directive, 32
    type alias, 32

virtual, 32
void, 32
volatile, 1, 33

wchar_t, 33
while, 1, 33

xor, 34
xor_eq, 34